RAG Based Citation Grounded Document Q&A System

Author - Shah Manav Amit
© Shah Manav Amit – All Rights Reserved
(DOI -10.5281/zenodo.22683839)

Abstract

Large Language Models (LLMs) are fluent but ungrounded: they generate plausible text without a mechanism to guarantee that any given claim is actually supported by a verifiable source, and they routinely hallucinate facts with high confidence. This project presents a Retrieval Augmented Generation (RAG) system for document intelligence that addresses this problem directly, rather than treating it as an afterthought. Users upload a dense, real world document, a policy, a research paper, or technical notes, and ask natural language questions about it. The system retrieves relevant passages using a hybrid of sparse (BM25) and dense (vector) search fused via Reciprocal Rank Fusion, generates an answer constrained to only the retrieved context, and then runs a dedicated claim level entailment check that verifies each individual citation actually supports the specific claim it is attached to, rather than merely confirming that a cited chunk was retrieved at all. The system was evaluated on a 14 question hand labelled QA set, achieving a 100% retrieval hit rate, a perfect Mean Reciprocal Rank, 100% refusal accuracy on unanswerable questions, and 80% citation level faithfulness after fixing three distinct verification bugs uncovered during real testing. The result is a working, end to end pipeline in which answers are not just cited, but citation verified.

Keywords

Retrieval Augmented Generation
Large Language Models
Hallucination Detection
Citation Verification
Hybrid Retrieval
BM25
Dense Passage Retrieval
Document Question Answering

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Introduction

Retrieval Augmented Generation combines a language model's fluency with an external knowledge source, allowing answers to be grounded in specific retrievable evidence rather than the model's parametric memory alone. Most RAG systems, however, stop at the point of producing a citation. Attaching a citation to an answer is not the same as confirming that the cited passage actually supports the specific claim being made. A language model can cite a real, retrieved chunk while still making a claim that chunk does not support.

The system was built as a solo, end to end project spanning document ingestion, chunking, embedding and indexing, hybrid retrieval, grounded generation, claim level hallucination verification, formal evaluation against a hand labelled QA set, and an interactive Streamlit interface.

Literature Survey

The system builds directly on four lines of prior work. Lewis et al. introduced the original Retrieval Augmented Generation architecture, combining a pre trained sequence to sequence generator with a non parametric dense retrieval index, and showed that conditioning generation on retrieved passages improves factuality and provides provenance that pure parametric models lack [1]. Karpukhin et al. proposed Dense Passage Retrieval (DPR), demonstrating that a simple dual encoder trained on question passage pairs substantially outperforms classical sparse retrieval methods for open domain question answering, establishing dense retrieval as a practical alternative to keyword search [2]. Robertson and Zaragoza formalised the Probabilistic Relevance Framework underlying BM25, the sparse ranking function this project fuses with dense retrieval, and documented its continued effectiveness relative to newer retrieval models [3]. Finally, Ji et al. surveyed the hallucination problem across natural language generation tasks, cataloguing detection and mitigation approaches and framing hallucination as a distinct, measurable failure mode rather than an unavoidable side effect of generation, a framing this project's claim level verification stage directly operationalises [4].

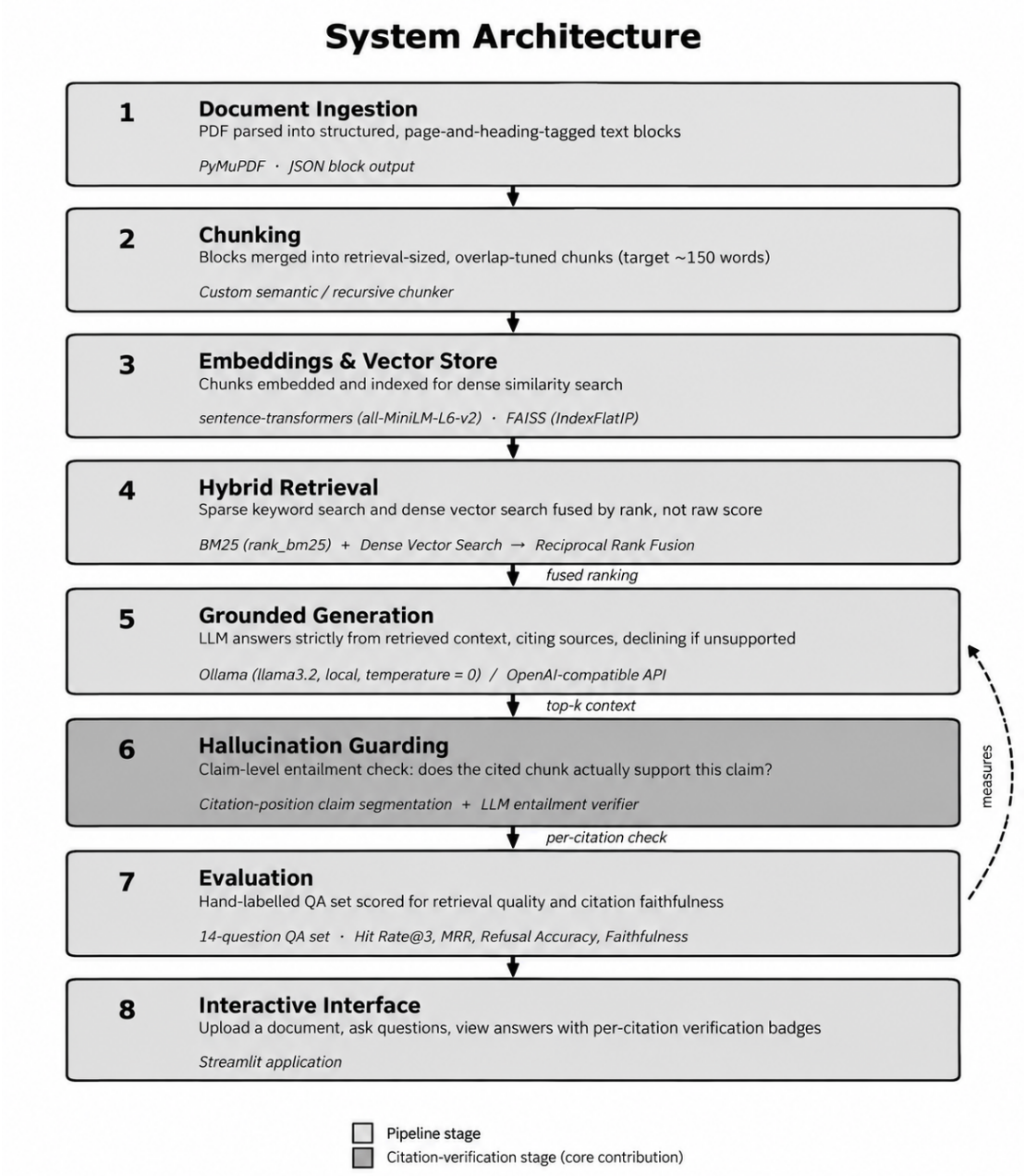
This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Taken together, this prior work motivates the specific design of this project: retrieval quality (informed by [2] and [3]) is treated as necessary but not sufficient, and generation faithfulness (informed by [1] and [4]) is treated as a property that must be explicitly measured at the level of individual claims, not assumed from the presence of a citation.

Methodology

The system is implemented as an eight phase pipeline, each phase independently verified against real output before the next was built.

(Figure 1 : System Architecture)



(Figure 1 : System Architecture)

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Phase 1 — Document Ingestion

Uploaded PDFs are parsed using PyMuPDF to extract raw per page text, producing structured JSON output for downstream processing.

Phase 2 — Chunking

Extracted text is split using semantic and recursive chunking with tuned overlap, balancing chunk coherence against retrieval granularity.

Phase 3 — Embeddings and Vector Store

Each chunk is embedded using sentence transformer models (bge small/large, or OpenAI/ Gemini embeddings as alternatives) and indexed in FAISS for approximate nearest neighbour retrieval.

Phase 4 — Hybrid Retrieval

Queries are matched using both BM25 sparse keyword search and dense vector similarity search, with results fused via Reciprocal Rank Fusion. Testing confirmed cases where hybrid fusion recovered correct answers that either method alone missed individually.

Phase 5 — Grounded Generation

An LLM (Ollama running llama3.2 locally by default, with an OpenAI compatible API as an alternative) is prompted to answer strictly from retrieved context, with inline citations back to specific source chunks, and to explicitly decline when the document does not contain an answer.

Phase 6 — Hallucination Guarding

A claim level entailment check verifies that each individual citation actually supports the specific claim it is attached to, rather than merely confirming the cited chunk exists. Three distinct bugs were identified and resolved during development: false negatives caused by brittle YES/NO response parsing, false positives caused by claims and citations being split incorrectly when a citation trailed after sentence punctuation, and mis verification of malformed citation strings, resolved respectively through more robust response parsing, citation position based claim segmentation, and an explicit “could not verify” flag for malformed citations.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Phase 7 — Evaluation

A 14 question hand labelled QA set (10 answerable questions spanning every document section, and 4 deliberately unanswerable questions) was used to measure retrieval quality and answer faithfulness. Diagnostic inspection of raw model and verifier output, rather than trusting the aggregate score alone, surfaced a further bug: the Ollama generation calls had no fixed temperature, unlike the already deterministic OpenAI path, which was resolved by pinning temperature to 0.0.

Phase 8 — User Interface

A Streamlit application wraps Phases 1 through 6, allowing a user to upload a PDF, ask a question, and view the generated answer with its supporting citations.

The modular design makes each stage independently testable, allowing errors to be identified and corrected without affecting the rest of the pipeline. This ensures the final system remains reliable, reproducible, and scalable for practical document based question answering.

Model Implementation and Configuration

Language Model -

The generation and verification stages both run on Llama 3.2, served locally through Ollama. Using a local, open-weight model keeps the pipeline fully free to run and avoids any dependency on external API availability or cost. An OpenAI-compatible API is supported as a drop-in alternative for both generation and verification, so the same prompts and parsing logic work against either backend without code changes. Temperature is pinned to 0.0 for both backends and this was not the case initially for the Ollama path, and fixing it (Phase 7) measurably improved faithfulness by removing sampling-induced inconsistency in citation behaviour.

Embeddings -

Chunks and queries are embedded using sentence-transformer models, primarily the bge-small/largefamily, with OpenAI or Gemini embeddings supported as alternatives. This keeps the embedding step interchangeable with the generation backend.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Vector Store -

Embeddings are indexed in FAISS, using IndexFlatIP for exact inner-product similarity search over the chunk embeddings. This gives exact (not approximate) nearest-neighbour retrieval at the scale of a single document's chunk set, which is appropriate given the target use case (one document at a time) rather than a web-scale corpus.

Sparse Retrieval -

In parallel with dense search, BM25 (via rank_bm25) performs classical keyword-based sparse retrieval over the same chunk set, grounded in the Probabilistic Relevance Framework [3].

Fusion _ Reciprocal Rank Fusion (RRF) -

The dense and sparse result lists are fused by rank, not raw score, using Reciprocal Rank Fusion. This avoids the problem of combining two retrieval methods whose similarity scores live on different, non-comparable scales. Testing confirmed concrete cases where fusion recovered a correct answer that either BM25 or dense search missed individually — this is not a theoretical justification only, but an empirically observed effect during development.

Grounded Generation -

The top-k fused context is passed to the LLM with a prompt constraining it to answer strictly from the retrieved context, to attach inline citations back to specific source chunks, and to explicitly decline to answer when the context does not support one.

Hallucination Guarding -

A second, dedicated LLM call performs claim-level entailment checking: for each individual claim-citation pair in the generated answer, it verifies whether the specific cited chunk actually supports that specific claim — not merely that the chunk exists or was retrieved. This stage is implemented via citation-position-based claim segmentation (splitting the answer into claims based on where citations appear, rather than by naive sentence splitting) feeding into an LLM entailment verifier.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Technology Stack

Layer	Technology / Library	Purpose
Document parsing	PyMuPDF	Extracts raw text per page from uploaded PDFs
Chunking	Custom semantic / recursive chunker	Splits documents into overlap-tuned chunks for retrieval
Embeddings	Sentence-transformers (bge-small/large) / OpenAI / Gemini embeddings	Converts chunks and queries into dense vectors
Vector store	FAISS	Stores and searches dense chunk embeddings
Keyword retrieval	BM25	Sparse lexical retrieval, fused with dense search via Reciprocal Rank Fusion
Generation	Ollama (llama3.2, local) / OpenAI-compatible API	Produces answers constrained to retrieved context, with citations
Hallucination guard	Custom claim-level entailment checker	Verifies each citation actually supports its specific claim
Interface	Streamlit	Upload, ask, and view grounded, verified answers
Language / environment	Python, virtual environment	Core implementation language and isolation

Gap Addressed

This project addresses the gap between retrieving relevant information and actually verifying that the generated claims are supported by their citations. Unlike conventional RAG systems that stop after attaching citations, this system performs claim level citation verification, making citation faithfulness an explicit and measurable part of the pipeline.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Implementation and Results

All eight phases were implemented and verified end to end. Retrieval and generation were confirmed against real queries at each phase, and the hallucination guarding stage was specifically stress tested against real failure cases rather than synthetic ones, with each discovered bug fixed and re verified against ground truth before proceeding. The completed system was further validated live against two structurally different real documents, a formal research paper and a set of college data structures notes, both correctly producing citation grounded answers, indicating the pipeline generalises beyond a single document type or domain.

Step 1 — Ingestion (once, at upload time)

The user uploads a PDF through the Streamlit interface. PyMuPDF parses it into structured, page-and-heading-tagged text blocks, emitted as JSON.

Step 2 — Chunking (once, at upload time)

The custom semantic/recursive chunker merges these blocks into overlap-tuned chunks targeting ~150 words each, balancing enough context per chunk against fine enough retrieval granularity.

Step 3 — Indexing (once, at upload time)

Each chunk is embedded via the sentence-transformer model and written into a FAISS IndexFlatIP index. The same chunks are also indexed for BM25 keyword search. From this point, the document is query-ready.

Step 4 — Query submission

The user types a natural-language question into the Streamlit UI.

Step 5 — Hybrid Retrieval

The query is simultaneously:

- Embedded and matched against the FAISS index (dense similarity search), and
- Matched against the BM25 index (sparse keyword search).

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Step 6 — Grounded Generation

The top-k chunks are passed to Llama 3.2 (via Ollama, temperature 0.0) with a prompt instructing it to answer using only this context, cite the specific chunk(s) supporting each claim inline, and explicitly decline if the context doesn't contain an answer.

Step 7 — Hallucination Guarding

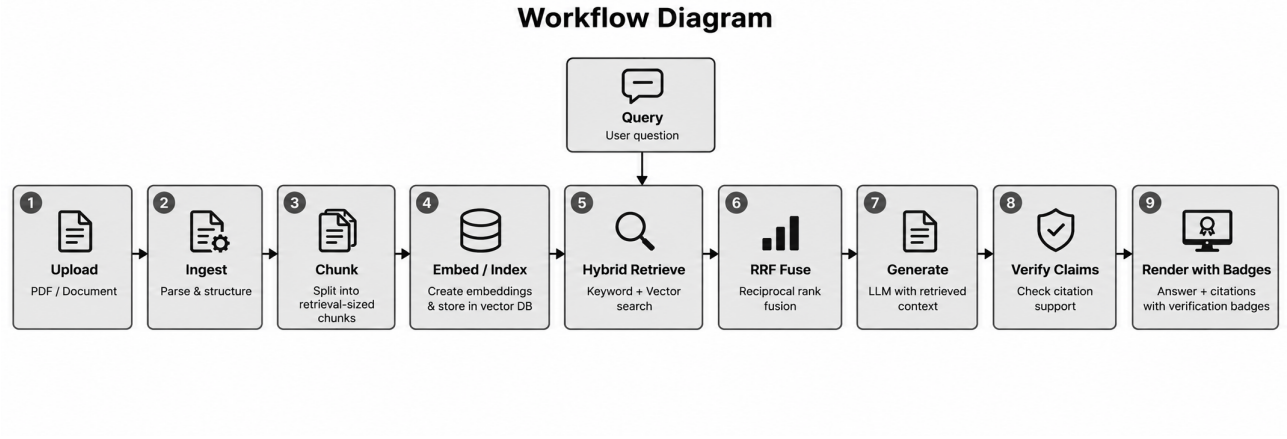
The generated answer is segmented into individual claim-citation pairs using citation-position-based segmentation. For each pair, a second LLM call performs an entailment check: does the specific cited chunk actually entail this specific claim? Each pair is marked verified, unsupported, or "could not verify" (for malformed citations).

Step 8 — Presentation

The Streamlit interface renders the final answer with a per-citation verification badge next to each claim, so the user can see at a glance which parts of the answer are citation-verified versus flagged.

Step 9 — Refusal path

If retrieval returns no chunks relevant enough to support an answer (as tested against the 4 deliberately unanswerable questions in the evaluation set), the model is expected to decline rather than fall back on general knowledge — this refusal behavior is itself scored in evaluation (Refusal Accuracy).



(Figure 2 : Workflow Diagram)

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Metric	Score
Retrieval Hit Rate@3	100%
Mean Reciprocal Rank (MRR)	1.000
Refusal accuracy (unanswerable questions)	100%
Citation structural validity	100%
Citation faithfulness (claim-level entailment)	80% (8/10)

The 80% citation faithfulness score reflects genuine remaining verifier limitations rather than a measurement artefact. Manual inspection of the underlying failures confirmed one true spurious citation catch and one residual false negative, which is documented as an open item for future refinement rather than treated as resolved.

Key Contributions and Novelty

• Citation faithfulness is treated as a first class, measured property of the system, not an assumed side effect of retrieval. The majority of the system's phases exist specifically to detect and quantify citation level hallucination.

• Hybrid retrieval (BM25 + dense vectors via Reciprocal Rank Fusion) is empirically shown to recover correct answers that either retrieval method misses alone.

• The system explicitly refuses to answer when a document does not contain the requested information, rather than defaulting to the model's general knowledge.

• Every reported bug fix in this project was found through inspection of real model output on real queries, not assumed from aggregate metrics, including a deterministic inference fix (pinning LLM temperature) that measurably improved faithfulness.

• The pipeline is provider agnostic: it runs entirely on free, local inference (Ollama) by default, with an OpenAI compatible API as a drop in alternative.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

Future Scope

• Closing the remaining citation faithfulness gap by improving the entailment verifier, potentially using a stronger dedicated verification model.

• Hardening the pipeline against edge cases such as empty PDFs, scanned or image only documents, and very large files.

• Pinning exact dependency versions in requirements.txt for full reproducibility.

• Extending evaluation to a larger and more diverse hand labelled QA set across additional document domains.

• Adding multi document and cross document question answering.

Conclusion

This project demonstrates a complete, working Retrieval Augmented Generation pipeline in which citation faithfulness is explicitly engineered for and measured, not assumed. Across eight independently verified phases, the system achieves strong retrieval performance, perfect refusal accuracy on unanswerable questions, and meaningfully improved, though not yet perfect, citation faithfulness, with every major limitation found and fixed through direct inspection of real failures rather than trust in aggregate scores. The system is functional end to end and has been validated on real, structurally distinct documents, meeting its original goal of producing answers that are not just cited, but citation verified.

Project Repository

Github Link : https://github.com/shahmanavamit-lang/RAG-Citation-System.git

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

References

[1] P. Lewis et al., “Retrieval augmented generation for knowledge intensive NLP tasks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[2] V. Karpukhin et al., “Dense passage retrieval for open domain question answering,” in Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP), Online, 2020, pp. 6769–6781.

[3] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[4] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Computing Surveys, vol. 55, no. 12, Art. 248, pp. 1–38, Dec. 2023.

© Shah Manav Amit – All Rights Reserved
(DOI -10.5281/zenodo.22683839)

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).
To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-nd/4.0/